

Lab: BFS and DFS Python Coding

Artificial Intelligence

Name _____ Date _____

Purpose

In this step-by-step lab, you will be simulating the process of DFS and BFS using the Python programming language.

Instructions

1. Set Up GitHub Codespace

Set up your own GitHub account using your school email address

- Visit <https://github.com/> and click Sign Up - then follow directions
- Visit <https://github.com/codespaces>
- Select [NEW CODESPACE]
- Under **repository** enter/paste: *MrTomaino/DFS-and-BFS*
- Make sure you install the Python extension

2. Following a graph by hand and with code - Depth First Traversal

- Load "Directional Graph A through I.png"
- Visually, follow a **depth first traversal** - visiting ALL of the nodes along the way (always visit the right hand node first, utilize explored set).
- What is the order that you visit each node:

A C G F I E H B D

- Switch to "main-graphPath.py"
- Uncomment (remove the # from) lines 14-18 to reveal the DFS visit code
- Click the RUN icon
- What is the order that the system traversed the graph?

A C G F I E H B D

Now, examine the DFS code by loading the **dfsVisit.py** document.

3. Following a graph by hand and with code - Breadth First Traversal

- a. Load “Directional Graph A through I.png”
- b. Visually, follow a **breadth first traversal** - visiting ALL of the nodes along the way (always visit the right hand node first, utilize explored set).
- c. What is the order that you visit each node:

A C B G F E D I H

- d. Switch to “main-graphPath.py”
- e. Comment lines 14-18 to hide the DFS visit code
- f. Uncomment lines 20-23 to reveal the BFS visit code
- g. Click the RUN icon
- h. What is the order that the system traversed the graph?

A B C D E F G H I

Now, examine the DFS code by loading the **bfsVisit.py** document.

- i. Switching between *bfsVisit.py* and *dfsVisit.py* code tabs, what is the major difference between the data structure used in each algorithm?

Bfs uses FIFO and DFS uses LIFO

Other than the variable names, what is the instruction that is different and what is the effect of the difference?

It changes how nodes come off the frontier

4. Following a graph by hand and with code - Depth First Search

- a. Load “Directional Graph A through I.png”
- b. Visually, follow a **depth first search**
 - i. Initial state: A
 - ii. Goal state: H
 - iii. (always visit the right hand node first, utilize explored set).
- c. What is the solution you came to with this algorithm

['A', 'C', 'F', 'E', 'H']

- d. Switch to “main-graphPath.py”
- e. Comment lines 20-23 to hide BFS Visit code
- f. Uncomment lines 29-33 to reveal dfsPath code
- g. Click the RUN icon

- h. What is the solution that the code came up with?

A C G F I E H

- i. Is this solution optimal?

No the depth first search takes 4 edges while the shortest path possible is 3

Now, examine the DFS code by loading the **dfsPath.py** document. Compare it to dfsVisit.py - notice the increased complexity. Why is finding the path more complex than simply visiting each node?

5. Following a graph by hand and with code - Breadth First Search

- a. Load “Directional Graph A through I.png”
- b. Visually, follow a **breadth first search**
 - i. Initial state: A
 - ii. Goal state: H
 - iii. (always visit the right hand node first, utilize explored set).
- c. What is the solution you came to with this algorithm

['A', 'B', 'E', 'H']

- d. Switch to “main-graphPath.py”
- e. Comment lines 29-33 to hide DFS Path code
- f. Uncomment lines 35-39 to reveal bfsShortestPath code
- g. Click the RUN icon
- h. What is the solution that the code came up with?

['A', 'B', 'E', 'H']

- i. Is this solution optimal?

Yes, Breadth first search explores all the paths before determining the optimal

Now, examine the DFS code by loading the **bfsShortestPath.py** document. Compare it to dfsPath.py - What is the major difference?

6. Applying the algorithm to a Street Map

- a. Load “East Coast Cities.png”
- b. Visually, follow an intuitive search

- i. Initial state: Boston
- ii. Goal state: Miami
- c. What is the solution you came to with

['Boston', 'New York', 'Philadelphia', 'Baltimore', 'Washington, D.C.', 'Richmond', 'Raleigh', 'Jacksonville', 'Miami']

- d. Switch to “main-cityPath.py”
- e. Click the RUN icon
- f. What is the solution that the code came up with?

['Boston', 'New York', 'Philadelphia', 'Virginia Beach', 'Richmond', 'Raleigh', 'Jacksonville', 'Miami']

- g. Is this solution optimal?

No, it takes unnecessary detours through Virginia Beach and Richmon

Now, examine the main-cityPath.py code. What algorithm is used?

Depth First Search

Modify the code to use a better algorithm to find the optimal solution. What change did you make?

I changed algorithms to Breadth first search

What is the optimal solution?

['Boston', 'New York', 'Philadelphia', 'Virginia Beach', 'Raleigh', 'Jacksonville', 'Miami']